

Meta ads efficiency engine

Harshitha Akkineni

Contents

1	End-to-end summary	What the system does, why it is guarded, and what the simulator showed.
2	Problem statement and understanding	Why this is a decision-quality problem, not just a ROAS dashboard problem.
3	Assumptions	The simulated brand, campaign stages, ad types, and business constraints.
4	Dataset source and insights	What data was used, where it came from, what the engine can and cannot know.
5	Approach and rejected alternatives	Why ROAS sorting, an LLM media buyer, MMM-first, and auto-execution were rejected.
6	Recommendation algorithm	The full decision flow for stop, decrease, increase, hold, and create recommendations.
7	Dataset examples and test cases	Concrete simulated rows used to validate stop, hold, and increase behavior.
8	Failed test cases and evaluation	What failed cases reveal about naive ROAS rules and the engine's conservative tradeoff.
9	Architecture	The system diagram, AI sidecar, audit log, and failure-handling path.
10	AI boundary and production readiness	Where AI is used, where it is blocked, interface design, security, and deployment questions.

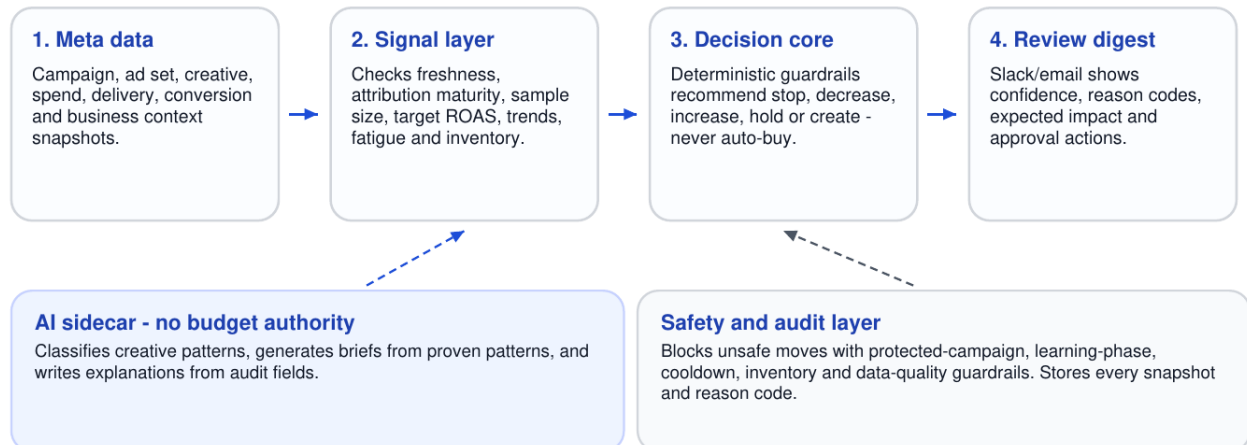
1. End-to-end summary

Section overview: The proposed system is a guarded recommendation engine for Meta ads. It recommends stop, decrease, increase, hold, and create actions, but keeps human approval in the loop.

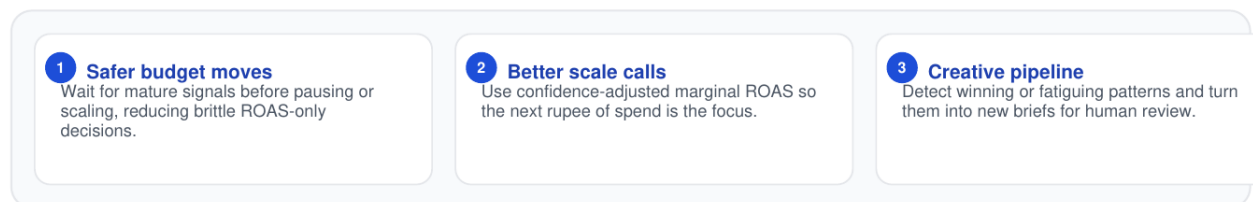
System diagram: guarded Meta ads efficiency engine

Recommendation-only v1

The system converts Meta performance data into safe, explainable recommendations while keeping budget authority with humans.



What this lets the team achieve



Core principle: do not optimize historical average ROAS alone - estimate whether the next unit of spend is likely to work.

Visual overview of the recommendation-only v1 system: Meta data becomes scored recommendations, AI stays outside budget authority, and Slack/email is used for human review.

The assignment is to improve ROAS for a brand running close to 10,000 Meta campaigns a month across Facebook and Instagram. I treated this as a decision system problem, not a reporting problem. At that scale, the hard part is not calculating ROAS. The hard part is deciding which ROAS numbers are mature enough to trust and which budget movements are safe to recommend.

The solution is a daily recommendation engine. It ingests Meta-style campaign, ad set, creative, spend, and conversion data; checks whether the signal is mature; estimates confidence-adjusted marginal ROAS; applies business guardrails; and sends human-reviewable recommendations through Slack or email.

Core thesis: Do not optimize historical average ROAS in isolation. Estimate whether the next rupee of spend is likely to work, then apply maturity, attribution, fatigue, inventory, learning-phase, and approval guardrails before recommending a budget move.

1.5%

Engine false pause rate

Compared with 16.8% for a simple ROAS rule.

5.4%

Engine wrong scale rate

Compared with 8.2% for a simple ROAS rule.

34.8%

Engine missed scale rate

Higher by design because v1 is conservative.

2. Problem statement and understanding

Section overview: The business problem is not that the team lacks ROAS. The problem is that manual judgment does not scale to 10,000 campaigns a month.

The straightforward reading of the assignment is: improve ad efficiency and ROAS. My interpretation is more specific: build a system that improves the quality of daily campaign decisions. A useful system should not merely rank ads by ROAS. It should decide whether each ad has been tested fairly, whether attribution has matured, whether the audience is saturating, and whether the next rupee of spend is likely to return more than the business target.

A pure ROAS rule is unreliable because Meta performance data is noisy. A small ad can show great ROAS after three lucky purchases. A young ad can look weak because delayed conversions have not arrived yet. A retargeting ad can look stronger than it really is because the audience already intended to purchase. A historically good ad can become a bad scale candidate when frequency rises and marginal returns flatten.

The system therefore needs to answer four operating questions every day. Which ads should be stopped because they have had a fair test and are unlikely to recover? Which budgets should be decreased because incremental spend is likely to perform worse than alternatives? Which budgets should be increased because marginal ROAS is still above target? Which new ads should be created because a winning creative pattern is under-covered or close to fatigue?

What the system optimizes

It optimizes expected incremental value from the next budget decision, not historical average ROAS alone.

What the system protects against

It protects against stopping recoverable ads too early and scaling noisy winners before the signal is mature.

Action	Meaning in the system
Stop	The ad has mature signal, is below target, and shows fatigue or saturation.
Decrease	The ad is not necessarily dead, but the next rupee is likely better used elsewhere.
Increase	The ad has mature signal, healthy frequency, and marginal ROAS above target.
Create	A winning product, audience, or creative pattern is under-covered or fatiguing.
Hold	The signal is not mature enough or business constraints block action.

3. Assumptions

Section overview: No real customer account or dataset was provided, so the solution uses explicit assumptions about the brand, ad types, campaign stages, and economics.

I assumed the customer is a multi-category consumer brand selling through Facebook and Instagram ads. I did not assume one product because the assignment mentions campaign scale, not a specific industry. A multi-category D2C account is more useful for testing because it creates different margins, conversion delays, creative formats, and audience dynamics.

The simulated product categories are beauty and skincare, apparel, home and lifestyle, electronics accessories, and supplements. These categories were chosen because they behave differently in Meta advertising. Skincare may convert quickly but has stricter claim constraints. Apparel is often creative-led and promotion-sensitive. Home and electronics accessories can have different price points and longer consideration windows. Supplements can look strong in direct response but require careful claim safety.

The simulated campaign stages are prospecting, retargeting, and retention. I kept these separate because their ROAS numbers are not directly comparable. Retargeting often looks stronger inside ad platforms because the audience already has intent. Prospecting may look weaker on immediate ROAS but can be more important for new customer growth.

Assumption	How it is represented	Why it matters
Brand type	Multi-category consumer brand	More realistic than one generic product when testing 10,000 campaigns.
Campaign stages	Prospecting, retargeting, retention	Different stages need different confidence and attribution treatment.
Creative formats	UGC reel, static image, carousel, demo, testimonial, comparison, offer-led ad	Needed for the create-new-ads recommendation.
Economics	Product-level margins and target ROAS assumptions	Budget decisions should optimize business economics, not one universal threshold.
Execution mode	Recommendation-only in v1	Human approval reduces risk before any Meta writeback is enabled.

Important: These assumptions are not presented as facts about a real customer account. They are the operating assumptions used to make the assignment testable without access to customer data.

4. Dataset source and insights

Section overview: The dataset used for evaluation is synthetic. It is designed to resemble Meta ad-account data and includes hidden truth so recommendations can be scored.

No real account dataset was provided. I therefore built a simulator instead of only describing an architecture. The simulated dataset contains observed fields that the engine can see and hidden fields that only the evaluator can see. This matters because real ad optimization is counterfactual: to know whether a stop or scale recommendation was good, we need some estimate of what would have happened otherwise.

The observed layer resembles what a Meta operator would usually inspect: campaign stage, ad age, spend, impressions, clicks, purchases, revenue, CTR, CPC, CPM, frequency, raw ROAS, and creative format. The hidden layer contains true marginal ROAS and future ROAS. The engine never sees the hidden layer. It is used only for evaluation.

Data source	Role in the project	Limit
Simulated account data	Main evaluation dataset for 10,000 campaigns and 120,000 ads.	Useful for testing logic, but not a live lift claim.
Meta-style schema assumptions	Field design for spend, impressions, clicks, conversions, revenue, and creative metadata.	Schema sanity only; not customer performance data.
Hidden ground truth in simulator	Lets the evaluator score false pauses, wrong scales, and missed scales.	Only available because this is a simulator.

Key dataset insights used by the engine

- Low spend can create fake winners because one or two purchases can produce very high ROAS.
- Young ads can create fake losers because conversions may arrive after the first few days.
- Retargeting can inflate platform ROAS because some users would have purchased anyway.
- High frequency and CTR decline are useful warning signs for fatigue and saturation.
- Average ROAS and marginal ROAS can disagree; budget decisions should care more about marginal ROAS.

Campaigns

10,000

Simulated monthly scale.

Ads

120,000

Used for recommendation testing.

Mature ads

84,822

Eligible for stronger decisions.

5. Approach and rejected alternatives

Section overview: I rejected approaches that are easy to build but unsafe for financial decisions. The chosen approach is guarded, explainable, and recommendation-first.

The simplest approach is to sort by ROAS. I rejected it because ROAS is noisy at low spend, delayed by attribution windows, inflated by retargeting overlap, distorted by promotions, and backward-looking. It moves money too confidently on incomplete signal.

I also rejected using an LLM as a media buyer. AI can summarize, explain, classify creatives, and draft new creative briefs. It should not decide whether money moves. Budget decisions need deterministic rules, reason codes, confidence, and an audit trail.

A full marketing mix model was not the right first version either. MMM-style thinking is useful for saturation and diminishing returns, but a full MMM-first approach is too heavy for daily ad-level recommendations and needs longer cross-channel history than the assignment provides.

I also rejected automatic execution on day one. The customer has access to APIs and MCP-style tools, so automation may be possible later. But the first version should recommend and explain. Execution should come after the system proves that its confidence scores are calibrated.

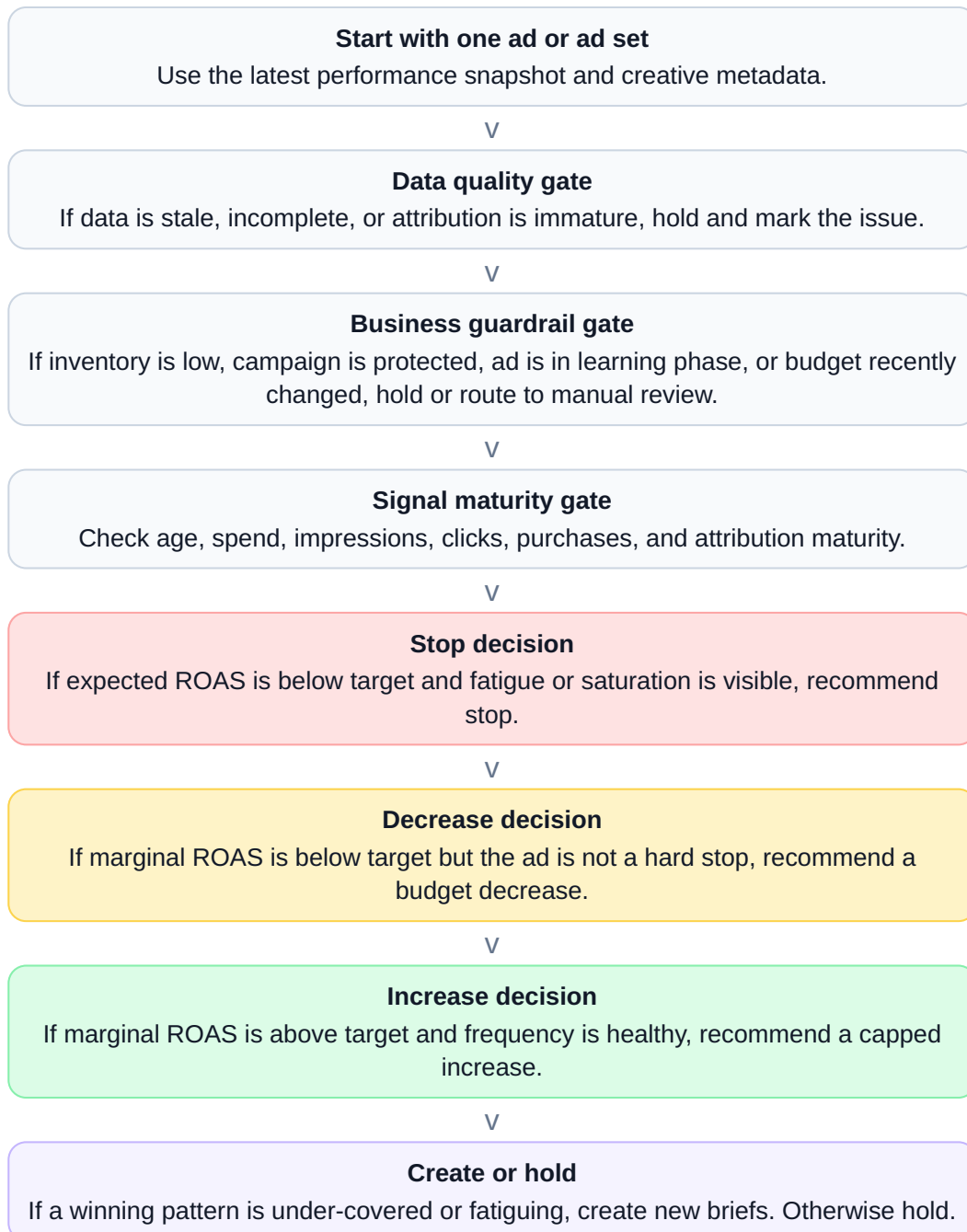
Alternative	Why it is tempting	Why I rejected it for v1
Pure ROAS sorting	Fast, simple, easy to explain.	Too brittle under low spend, delayed attribution, and retargeting inflation.
LLM media buyer	Looks powerful and flexible.	Unsafe for direct budget decisions and hard to audit.
MMM-first	Good for strategic spend-response questions.	Too slow and heavy for daily ad-level action.
Auto-execution first	Maximizes automation.	Too risky before confidence calibration and approval controls exist.

Final choice: A guarded recommendation engine that is deterministic for financial decisions, AI-assisted for explanation and creative work, and human-approved before action.

6. Recommendation algorithm

Section overview: The algorithm is a gated decision flow. It blocks action when data is weak and only recommends budget movement after signal maturity, economics, fatigue, and product-specific target ROAS are checked. Target ROAS is not hardcoded; it comes from product economics.

Algorithm flowchart



7. Dataset examples and test cases

Section overview: These examples are taken from the simulated dataset. They show why the engine does not treat every high-ROAS ad as a scale candidate or every low-ROAS ad as a stop candidate.

Case	Observed dataset row	Engine output	Why the output is correct
Stop	ad_000000 Home UGC Reel Prospecting spend INR 161,607 raw ROAS 1.25 target ROAS 2.62 marginal ROAS 0.90 frequency 11.5 CTR decline 65%	Stop	The ad is mature, below target, and saturated. The decision is not based on low ROAS alone.
Hold	ad_000900 Apparel UGC Reel Retargeting age 5 days spend INR 6,491 raw ROAS 7.43 true marginal ROAS 1.96 target ROAS 2.12	Hold / watchlist	The ad looks exciting, but it is too young and attribution-immature. Scaling would have been wrong.
Increase	ad_000002 Beauty Carousel Retargeting raw ROAS 3.00 expected ROAS 3.50 marginal ROAS 3.06 target ROAS 1.90 frequency 0.7	Increase with cap	The ad is mature, marginal return is above target, and frequency does not show saturation.

Pass test cases

Test case	Expected behavior	What it proves
TC-01: Mature fatigued loser	Stop ad_000000.	The engine can stop mature underperformance when fatigue and saturation are visible.
TC-02: Immature high-ROAS ad	Hold ad_000900.	The engine does not scale a tempting ad before signal and attribution mature.
TC-03: Mature scale candidate	Increase ad_000002 with a cap.	The engine can scale when marginal return is above target and saturation is low.

8. Failed test cases and evaluation

Section overview: Failed cases show what the system is protecting against. They also show the engine's main tradeoff: it is intentionally conservative in v1: it waits for stronger evidence before scaling, which can skip some good opportunities, but reduces costly wrong moves.

Failed case	Dataset row	What failed	What it shows
F-01: Baseline false pause	ad_000070 raw ROAS 1.26 target 1.90 age 3 days	A simple ROAS rule would stop an ad that hidden future performance says should not be stopped.	Average ROAS alone is too brittle. Maturity and attribution gates are necessary.
F-02: Baseline wrong scale	ad_000004 raw ROAS 2.31 true marginal ROAS 1.06 target 1.62	A simple ROAS rule would increase spend even though true marginal ROAS is below target.	High platform ROAS can come from retargeting inflation, noise, or small-sample luck.
F-03: Engine missed scale	ad_000012 true marginal ROAS 2.19 target 1.62 frequency 3.43	The guarded engine holds a row that hidden truth says could scale.	The engine is safer but conservative. Next iteration should tune thresholds and use controlled scale tests.

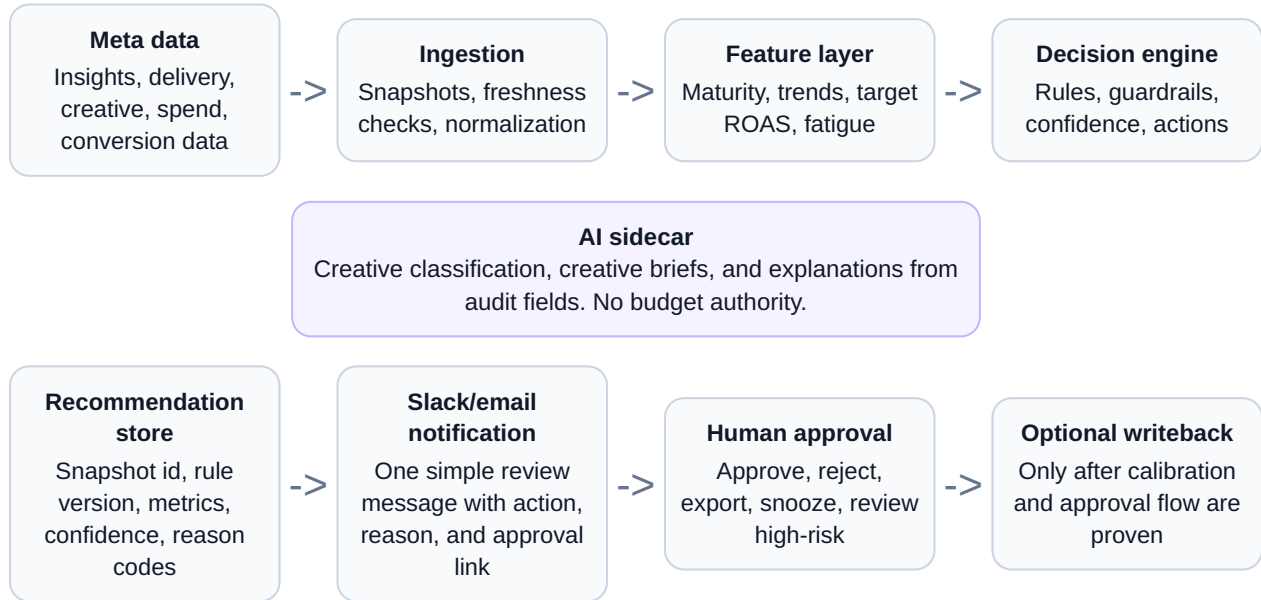
Metric	Simple ROAS rule	Guarded engine	Read
False pause rate	16.8%	1.5%	Fewer ads stopped before recovery.
Wrong scale rate	8.2%	5.4%	Fewer noisy or inflated ads scaled.
Missed scale rate	10.2%	34.8%	The guarded engine is more conservative in v1.
Stop recommendations	22,026	9,888	Fewer hard pauses.
Increase recommendations	83,193	39,703	Fewer scale calls because maturity and confidence are required.

The simulator is not a claim of live revenue lift. It is a test harness for known failure modes. Real validation would require account data, margin data, inventory data, and eventually an online holdout or lift test.

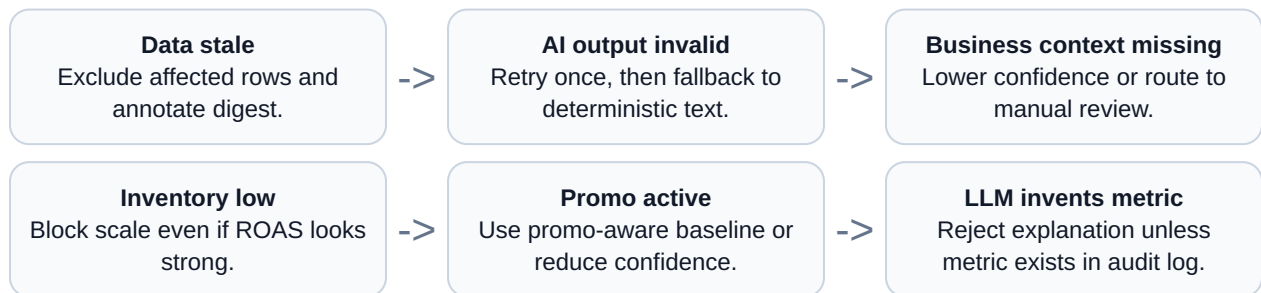
9. Architecture

Section overview: The architecture keeps the deterministic decision engine separate from the AI layer. If AI fails, recommendations still render. If data fails, action is blocked.

System architecture diagram



Failure-handling diagram



10. AI boundary and production readiness

Section overview: AI is useful for creative interpretation and explanation, but financial control stays deterministic and auditable.

AI is used for

- Classifying creative patterns such as UGC testimonial, product demo, comparison, routine simplification, or offer-led ad.
- Generating creative briefs from proven product, audience, and creative patterns.
- Writing Slack or email explanations from stored audit fields.

AI is not used for

- Deciding whether to stop, decrease, increase, or hold.
- Predicting ROAS directly.
- Executing changes in Meta.
- Inventing metrics or overriding guardrails.

Simple Slack / email notification

Subject: Meta ads recommendation - review 3 actions

Highest-impact action: STOP ad_000000

Reason: Mature underperformance with high frequency, CTR decline, and marginal ROAS below target.

Also flagged: 1 capped scale candidate and 1 watchlist item.

Action: Open dashboard to approve, reject, or snooze.

Production readiness

The first version should use read-only Meta access. Write credentials should be separate and enabled only after the approval system exists. Secrets should live in a secret manager. The LLM should receive aggregated metrics and creative text, not user-level purchase logs, customer emails, phone numbers, payment data, or raw customer identifiers.

Before deployment, I would confirm the real objective, margin and refund data, inventory and promo context, attribution window, protected campaigns, and the daily budget movement threshold that requires human approval.

Closing position: The right first version is not a fully autonomous ad buyer. It is a reliable recommendation layer that can be inspected, challenged, and calibrated before low-risk execution is automated.